OPERATING SYSTEMS LABORATORY

DEADLOCK AVOIDANCE USING BANKER'S ALGORITHM

Practical Record / Experiment

Student Name	_____
Roll Number	_____
Class / Division	_____
Subject	Operating Systems
Experiment No.	_____
Date	_____

Experiment scope: This practical implements the safety-checking part of Banker's Algorithm using simple one-dimensional arrays for exactly three resource types (A, B and C). It determines whether the supplied allocation state is safe and prints a safe sequence when one exists.

1. AIM

To implement Banker's Algorithm in C to check whether a resource-allocation state is safe and, if it is safe, display the safe sequence of processes.

2. TITLE

Deadlock Avoidance using Banker's Algorithm (using one-dimensional arrays for three resource types).

3. PROBLEM STATEMENT

Write a C program that accepts the number of processes, the number of resource types, the Allocation data, the Maximum demand data and the Available resources from the user. Calculate the Need values, apply the safety algorithm and display whether the system is in a safe state. If safe, print the order in which all processes can complete.

4. OBJECTIVES

• Understand deadlock and the idea of a safe state.
• Calculate the Need of each process using Maximum – Allocation.
• Implement the safety check of Banker's Algorithm.
• Identify whether all processes can finish in some order.
• Display a safe sequence when one exists.

5. THEORY

Deadlock: A situation in which a group of processes is blocked because each process is waiting for a resource held by another process in the group.

Deadlock avoidance: The operating system examines a possible resource allocation before granting it. It grants a request only if the resulting state remains safe.

Safe state: A state is safe if at least one order (safe sequence) exists in which every process can obtain its remaining required resources, finish, and release its allocated resources.

Banker's Algorithm terms

Term	Meaning
Allocation	Resources currently assigned to a process.
Maximum	Maximum total resources a process may claim.
Available	Resources currently free in the system.
Need	Additional resources required to finish.
Work	Temporary copy of Available, updated during the safety test.
Finish	Flag showing whether a process has been marked complete.

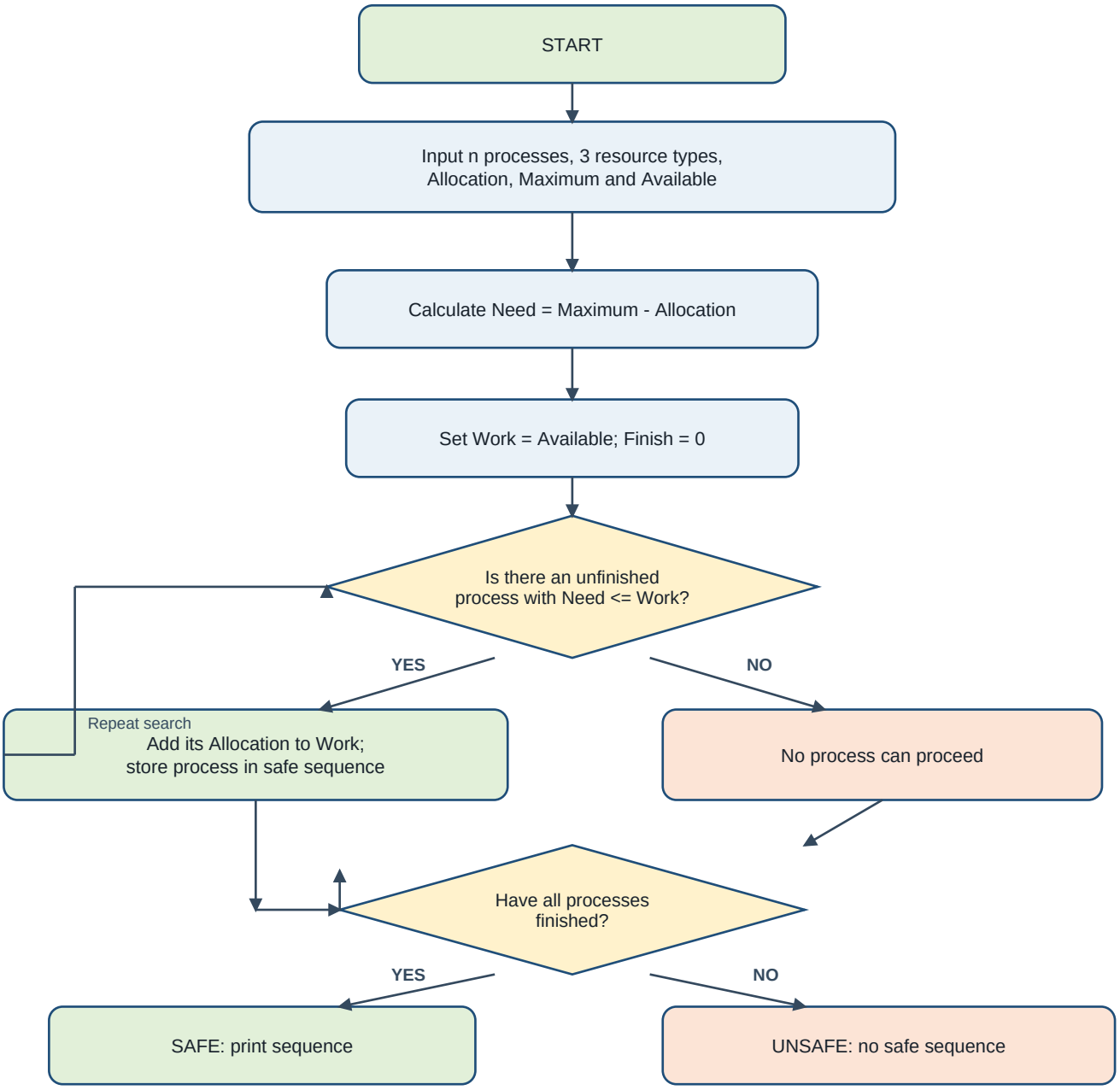
Formula: Need[i] = Maximum[i] – Allocation[i]. A process can be selected when each of its Need values is less than or equal to the corresponding Work value.

6. ALGORITHM / STEPS

1. Start the program.
2. Read the number of processes (n).
3. Read the number of resource types. This simplified program accepts exactly 3 types: A, B and C.
4. Read the Allocation values for each process.
5. Read the Maximum values for each process.
6. Read the Available values for A, B and C.
7. Calculate Need = Maximum – Allocation for every process.
8. Initialize Work = Available and mark every process as unfinished.
9. Find an unfinished process whose Need is less than or equal to Work for all three resources.
10. If found, add that process's Allocation to Work, mark it finished and add it to the safe sequence.
11. Repeat the search until all processes finish or no further process can be selected.
12. If all processes finish, print SAFE and the safe sequence; otherwise print NOT SAFE.
13. Stop.

7. FLOWCHART

The following flowchart shows the safety-checking logic.


START
Input n processes, 3 resource types,
Allocation, Maximum and Available
Calculate Need = Maximum - Allocation
Set Work = Available; Finish = 0
Is there an unfinished
process with Need <= Work?
YES
NO
Repeat search
Add its Allocation to Work;
store process in safe sequence
No process can proceed
Have all processes
finished?
YES
NO
SAFE: print sequence
UNSAFE: no safe sequence

8. C PROGRAM

The program uses separate one-dimensional arrays for resources A, B and C. It is intentionally limited to three resource types to keep the code easy to follow.

```
#include <stdio.h>

int main()
{
    int n, m, i, count = 0, found;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resource types: ");
    scanf("%d", &m);

    if (m != 3)
    {
        printf("This program supports 3 resource types (A, B, C).");
        return 0;
    }

    int A[n], B[n], C[n];
    int MA[n], MB[n], MC[n];
    int NA[n], NB[n], NC[n];
    int finish[n], safe[n];
    int a, b, c;

    printf("\\nEnter Allocation (A B C) for each process:\\n");
    for (i = 0; i < n; i++)
        scanf("%d %d %d", &A[i], &B[i], &C[i]);

    printf("\\nEnter Maximum (A B C) for each process:\\n");
    for (i = 0; i < n; i++)
        scanf("%d %d %d", &MA[i], &MB[i], &MC[i]);

    printf("\\nEnter Available (A B C): ");
    scanf("%d %d %d", &a, &b, &c);

    // Calculate Need and initialize Finish
    for (i = 0; i < n; i++)
    {
        NA[i] = MA[i] - A[i];
        NB[i] = MB[i] - B[i];
        NC[i] = MC[i] - C[i];
        finish[i] = 0;
    }

    // Banker's safety algorithm
    while (count < n)
    {
        found = 0;

        for (i = 0; i < n; i++)
        {
            if (finish[i] == 0 &&
                NA[i] <= a && NB[i] <= b && NC[i] <= c)
            {
                a = a + A[i];
                b = b + B[i];
                c = c + C[i];

                safe[count] = i;
                count++;
                finish[i] = 1;
                found = 1;
            }
        }

        if (found == 0)
            break;
    }

    // Display the result
    if (count == n)
    {
        printf("\\nSystem is in SAFE state.\\n");
        printf("Safe Sequence: ");

        for (i = 0; i < n; i++)
```

```
            printf("P%d ", safe[i]);
    }
    else
    {
        printf("\\nSystem is NOT in SAFE state.\\n");
        printf("Processes that could not finish: ");

        for (i = 0; i < n; i++)
            if (finish[i] == 0)
                printf("P%d ", i);
    }

    return 0;
}
```

9. SAMPLE INPUT

```
Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation (A B C) for each process:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Maximum (A B C) for each process:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available (A B C): 3 3 2
```

10. SAMPLE OUTPUT

```
System is in SAFE state.
Safe Sequence: P1 P3 P4 P0 P2
```

11. WORKING / EXAMPLE

For the sample input, the Need values are calculated as Maximum – Allocation:

Process	Allocation (A B C)	Maximum (A B C)	Need (A B C)
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Initial Available / Work: (3, 3, 2). P1 can finish because its Need (1, 2, 2) is less than or equal to Work. P1 releases its Allocation (2, 0, 0), so Work becomes (5, 3, 2).

The same check gives the following order:

Process completed	Work after it releases its Allocation
P1	(5, 3, 2)
P3	(7, 4, 3)
P4	(7, 4, 5)
P0	(7, 5, 5)

Process completed	Work after it releases its Allocation
P2	(10, 5, 7)

All five processes can finish. Therefore, the system is in a safe state and one safe sequence is P1 → P3 → P4 → P0 → P2.

12. STEPS TO WRITE THE PROGRAM IN A PRACTICAL EXAM

1. Write the header file and main function.
2. Declare n, m, loop variables and count.
3. Declare the one-dimensional arrays for Allocation, Maximum, Need, Finish and Safe Sequence.
4. Read n and m. Check that m is 3 for this version.
5. Read the three Allocation values for each process.
6. Read the three Maximum values for each process.
7. Read the three Available resource values.
8. Use a for loop to calculate the Need values and initialize Finish to 0.
9. Write the while loop for Banker's safety algorithm.
10. Inside it, search for an unfinished process whose three Need values are available.
11. Release its allocated resources into the available variables and store its process number in Safe.
12. If no process is found in one complete pass, stop the loop.
13. Print SAFE and the safe sequence if count equals n; otherwise print NOT SAFE and unfinished processes.
14. Compile, run and test with the sample input.

13. IMPORTANT NOTES

• This simplified program uses one-dimensional arrays and supports exactly three resource types (A, B and C). The input asks for m so that the program can check that the user entered 3; it does not support arbitrary m.
• Banker's Algorithm is a deadlock-avoidance method. The code above performs the safety check; it does not implement a general deadlock-detection algorithm or recovery procedure.
• An unsafe state does not necessarily mean that deadlock has already occurred. It means that the algorithm cannot find a safe sequence for the current state.
• For a general deadlock-detection algorithm, the system needs a current Request matrix (outstanding requests), which is different from the Maximum/Need data used by Banker's Algorithm.

14. CONCLUSION

The Banker's Algorithm was implemented in C using one-dimensional arrays for three resource types. The program accepts Allocation, Maximum and Available resource values, calculates the Need values and checks whether the system is in a safe state. For the given sample input, the system is safe and the safe sequence is P1 → P3 → P4 → P0 → P2.

15. RESULT

Thus, the program to check system safety using Banker's Algorithm was written, executed and verified successfully.

Viva reminder: Allocation = resources already assigned; Maximum = maximum claim; Available = free resources; Need = Maximum – Allocation.